The data use rules

- Participant-level data (one row per person) stays in the Workbench: no copying out, downloading, or screenshots.
- No count of 1–20 participants may be shared, directly or by calculation (differences, percentages). 0 is allowed. Allele counts under 40 are shown as `<40`.
- Downloads are monitored; participant-level downloads trigger an egress alert.
- Nothing in a GitHub repository (code, comments, outputs) may contain participant data or counts under 20.
- Notify the program 2 weeks before publishing or presenting results.

Articles at `support.researchallofus.org/hc/en-us/articles/`*number*: Data User Code of Conduct 22346176432532; Data and Statistics Dissemination Policy 22346276580372; Egress Alert Policy 4407354684052; Large Download Requests 48941559021716; Publication and Presentation Policy 22346271543828; Researcher FAQ 52861503885460. A workable habit: keep person-level files in one folder that never leaves the workspace, and aggregate outputs in another that is checked before any download.

Moving around the terminal

`pwd`	print the folder you are in
`ls -lh`	list files with sizes (`h` = human units)
`ls -lt | head`	newest files first
`cd my_project`	move into a folder
`cd ..` / `cd ~`	up one level / home folder
`mkdir -p results`	make a folder (no error if it exists)
`cp a b` / `mv a b`	copy / move or rename
`rm file`	delete a file (no trash can – careful)
`du -sh data`	total size of a folder
`df -h .`	free disk space on this VM
`history | tail -20`	your last 20 commands

Tab completes file names. Up-arrow recalls earlier commands. `Ctrl-C` stops a running command. Edit a file: `nano file`; save and exit with `Ctrl-O`, `Enter`, `Ctrl-X` (or, if that fails in a Windows browser, `Ctrl-X`, `Y`, `Enter`).

Looking inside a text file

`head -n 3 file.tsv`	first 3 lines (always look first)
`tail -n 3 file.tsv`	last 3 lines
`less -S file.tsv`	scroll through; `q` quits; `-S` no wrap
`wc -l file.tsv`	count lines (rows + 1 header)
`expand -t 14 file.tsv | head`	line up columns for reading
`head -1 file.tsv | tr '\t' '\n' | cat -n`	number the columns of the header
`cut -f 2,5 file.tsv | head`	keep columns 2 and 5 (tab-separated)
`cut -d, -f 2 file.csv | head`	same, comma-separated
`grep -c 'Female' file.tsv`	count lines containing a word
`grep -n 'NA' file.tsv | head`	show matching lines, with line numbers
`sort | uniq -c`	count each distinct value (after `cut`)
`awk -F'\t' 'NR>1{s+=$2} END{print s/(NR-1)}' f.tsv`	mean of column 2, skipping the header
`zcat file.gz | head`	look inside a gzipped file

Example, a frequency table of column 5 of a phenotype file: `cut -f 5 pheno.tsv | sort | uniq -c`. The sort is required: `uniq` only merges adjacent lines. Output stays on your screen; it is not a file to share.

Settings and environment variables

`source config.sh`	load the settings into this terminal: needed before any $NAME below, once per tab
`echo $CDR_DATASET`	v9 CDR: `wb-silky-artichoke-2408.C2025Q4R6`
`echo $WORKSHOP_BUCKET $BILLING_PROJECT`	bucket and billing project
`env | grep -i -E 'workspace|cdr|google'`	Workbench variables, if set

The Workbench can define `$CDR_DATASET`, `$WORKSPACE_BUCKET`, `$GOOGLE_PROJECT`, and `$CDR_STORAGE_PATH`, but they are not always set in a new terminal (the "Getting Started" featured-workspace notebooks set them up). A project's own settings file (`export NAME=value`, one per line, sourced by every script) does not depend on them. Set one value for one run with `NAME=value bash script.sh`.

The CDR: BigQuery from the terminal (bq)

`bq ls $CDR_DATASET | head -40`	list the CDR tables
`bq show -schema -format=prettyjson $CDR_DATASET.person`	column names and types of one table
`bq query -use_legacy_sql=false -dry_run '...'`	report bytes a query would scan; runs nothing, costs nothing
`bq query -use_legacy_sql=false -maximum_bytes_billed=25000000000 '...'`	run with a hard cost cap
`bq query ... -format=csv -max_rows=10000000 '...' > out.csv`	save results as CSV (person-level: keep it in the workspace)

Table names go in backticks inside SQL: `FROM \`$CDR_DATASET.measurement\``. Look up concept IDs in the public Data Browser.

Handy queries (aggregate only)

```
SELECT COUNT(DISTINCT person_id) FROM `$CDR_DATASET.measurement`
WHERE measurement_source_concept_id = 903133
SELECT concept_id, concept_name, vocabulary_id
FROM `$CDR_DATASET.concept`
WHERE LOWER(concept_name) LIKE '%height%' LIMIT 20
```

Cloud storage (gcloud storage; gsutil is the older tool)

`gcloud storage ls $WORKSHOP_BUCKET`	list a bucket or folder
`gcloud storage ls -l gs://.../file`	list with size
`gcloud storage du -s -r gs://.../folder`	total size of a cloud folder (-r readable units)
`gcloud storage cat gs://.../file.tsv | head -3`	peek at a cloud text file without copying it
`gcloud storage cp gs://.../file lab_data/`	copy cloud → VM disk
`gcloud storage cp results/x.png $WORKSHOP_BUCKET/lab/`	copy VM disk → a workspace bucket
`gcloud storage cp -r folder $WORKSHOP_BUCKET/lab/`	copy a whole folder
`gcloud -billing-project=$BILLING_PROJECT storage cp ...`	**requester-pays** buckets (e.g., the controlled dataset bucket): name who pays, before `storage`

Older equivalents: `gsutil ls`, `gsutil cp`, `gsutil -u $BILLING_PROJECT cp ...` (requester pays). An error mentioning "requester pays" or "billing" means the billing flag is missing.

Controlled Tier genomic data (v9)

Root: `gs://vwb-aou-datasets-controlled/v9/`. Below, `wgs/` means `wgs/short_read/snpindel/`. From the v9 Controlled Tier data dictionary; list a folder with `gcloud storage ls` for exact file names. All copies need the billing flag.

`wgs/acaf_threshold/plink_bed/`	whole-genome variants with allele frequency >1% or allele count >100; PLINK 1, one set per chromosome (chr22 `.bed` ≈ 276 GB)
`wgs/acaf_threshold/pgen/ bgen/ vcf/`	same variants, other formats (chr22 `.pgen` ≈ 25 GB)
`wgs/exome/ wgs/clinvar/`	smaller callsets, same layout
`wgs/aux/ancestry/ancestry_preds.tsv`	predicted ancestry, 16 PCs per person (`research_id` = `person_id`)
`wgs/aux/relatedness/`	related pairs (kinship > 0.1); samples to drop for an unrelated set
`microarray/plink/`	genotyping-array data, PLINK 1, all chromosomes in one set

Variant IDs in these files are `chr:pos:ref:alt` (GRCh38), not rsIDs, and FID is 0. All of Us publishes no HapMap3 subset: make one with `-extract` on a list of IDs matched by position and alleles, then `-update-name` to rsIDs (the `.pvar` may be `.zst`-compressed: `plink2 -zst-decompress`).

PLINK 2: looking at genotype files

`head -n 2 f.fam`	people: FID IID father mother sex pheno
`head -n 2 f.bim`	variants: chr ID cM position allele1 allele2
`wc -l < f.fam; wc -l < f.bim`	number of people; number of variants
`cut -f 1 f.bim | uniq -c`	variants per chromosome
`plink2 -bfile f -freq -out out/f`	allele frequencies → `.afreq`
`plink2 -bfile f -missing -out out/f`	missing-call rates per person / variant
`plink2 -bfile f -extract ids.txt -make-bed -out out/g`	keep only listed variant IDs
`plink2 -bfile f -keep ppl.txt -make-bed -out out/g`	keep only listed people (IID, or FID IID)
`plink2 -pgen f.pgen -pvar f.pvar -psam f.psam ...`	name the three input files separately (e.g. a mounted genotype file)
`plink2 -bfile f -score w.txt 1 2 3 header -out out/s`	polygenic score from a weight file
`grep -E 'processed|skipped' out/s.log`	how many weights matched

`.bed` is binary: never open it with `head` or `less`. PLINK writes a `.log` next to every output; read it.

Git: getting code onto the VM and keeping it current

`git clone https://github.com/USER/REPO.git`	copy a repository onto the VM
`git pull`	fetch and apply the latest changes
`git status`	what changed; what is untracked or ignored
`git diff`	line-by-line changes not yet committed
`git log -oneline | head`	recent commits
`git rev-parse -short HEAD`	the commit you are on (record it with results)
`git add scripts/x.py`	stage a file you intend to commit
`git commit -m "Describe the change"`	record staged changes
`git push`	send commits to GitHub (needs a token)
`git check-ignore -v config.sh`	confirm a file is ignored, and by which rule
`git ls-files`	list every file git tracks
`git restore file`	discard uncommitted edits to a file

A `.gitignore` for Workbench projects

```
config.sh   # filled settings
data/     # copied-in data
work/     # person-level files
results/  # outputs (review before sharing)
.ipynb_checkpoints/
```

Commit code, templates, and documentation only. Before committing a notebook, clear its outputs (JupyterLab: Edit → Clear All Outputs): a saved `.ipynb` stores its outputs. Check with `git diff -cached` before every `git push`.

GitHub CLI (`gh`) – on your own computer

`gh auth login`	sign in once
`gh repo create NAME -private -clone`	new repository
`gh repo clone USER/REPO`	clone by short name
`gh repo view -web`	open the repo in a browser

Getting aggregate results to your computer

1. Run your disclosure check on the output folder (no count of 1–20, directly or by subtraction; no person-level file). It must pass.
2. Open each file in `results/` and review it yourself.
3. In the JupyterLab file browser, right-click the file → **Download**.

Never download person-level files, copied-in data, or an archive that contains them.